

# When Agda met Vampire

Artjoms Šinkarovs and Michael Rawson

---

Jia-Wei Zhang

10/2026

# Motivation

---

# Background

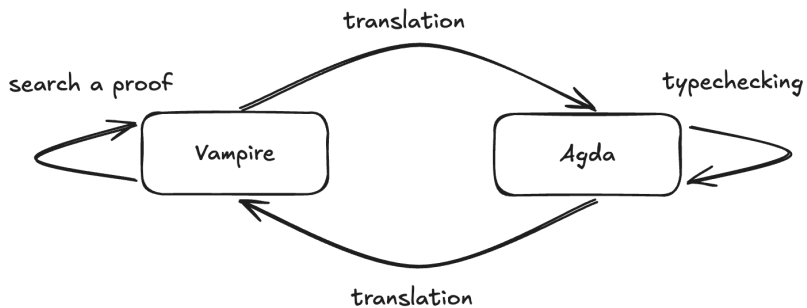
| ATP                                                                                                                                                                                                                   | ITP                                                                                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Automated Theorem Proving</p> <ul style="list-style-type: none"><li>▶ Machine searches for proof automatically</li><li>▶ ATPs are based on Classical FOL</li><li>▶ Modern ATPs: <b>E</b>, <b>Vampire</b></li></ul> | <p>Interactive Theorem Proving</p> <ul style="list-style-type: none"><li>▶ Human guides proof; machine checks every step</li><li>▶ Most ITPs are based on Intuitionistic Logic</li><li>▶ Proof Assistants: <b>Agda</b>, <b>Rocq</b>, <b>Lean</b></li></ul> |

Vampire, as the one of state-of-the-art ATP, has been under development for nearly 35 years.

Since 1999, Vampire won 78 titles in CASC.

Vampire won all competitions in 2025 and 2026!!

# Motivation



# Example 1: Vector Spaces (without Scalars)

```
postulate
  V      : Set
  ze     : V
  _+_    : V → V → V
  _-     : V → V

  neutl  : ∀ u → ze + u = u
  negl   : ∀ u → (- u) + u = ze

  assoc  : ∀ u v w →
    (u + v) + w = u + (v + w)

  ze-uniq' : ∀ u v →
    u + v = v → u = ze

-- (declare-sort V 0)
-- (declare-const ze V)
-- (declare-fun _+_ (V V) V)
-- (declare-fun _- (V) V)

-- (assert (! (forall ((u V))
--   (= (_+_ ze u) u)) :named neutl))
-- (assert (! (forall ((u V))
--   (= (_+_ (- u) u) ze))
--   :named negl))
-- (assert (! (forall ((u V) (v V) (w V))
--   (= (_+_ (_+_ u v) w)
--     (_+_ u (_+_ v w)))) :named assoc))
-- (declare-const u V)
-- (declare-const v V)
-- (assert (! (forall ()
--   (= (_+_ u v) v)) :named e))
-- (assert-not (! (= u ze) :named ze-uniq))
```

Agda to Vampire is based on **Agda reflection**.  
**Agda reflection** is built-in metaprogramming system.

# A Little Taste of Agda

```
trivial :  $\forall u \rightarrow (-u) + (u + u) = u$ 
trivial u = trans (trans t1 t2) (neutl u)
  where
    t1 :  $(-u) + (u + u) = ((-u) + u) + u$ 
    t1 = sym (assoc (-u) u u)

    t2 :  $((-u) + u) + u = ze + u$ 
    t2 = cong (_+ u) (negl u)
```

```
sym :  $\{x\ y : A\} \rightarrow x = y \rightarrow y = x$ 
cong :  $\{x\ y : A\} \rightarrow (f : A \rightarrow B) \rightarrow x = y \rightarrow fx = fy$ 
trans :  $\{x\ y\ z : A\} \rightarrow x = y \rightarrow y = z \rightarrow x = z$ 
neutl :  $\forall u \rightarrow ze + u = u$ 
negl :  $\forall u \rightarrow (-u) + u = ze$ 
assoc :  $\forall u\ v\ w \rightarrow (u + v) + w = u + (v + w)$ 
```

# Contents

Motivation

Classical FOL

Vampire Core Calculus

Vampire to Agda

Thesis Proposal

References

# Classical FOL

---

# Classical FOL

$x, y, z$  as individual variable.

$$\textit{term} ::= x \mid c \mid f(\textit{term}, \dots, \textit{term})$$
$$\textit{atom} ::= P(\textit{term}, \dots, \textit{term}) \mid \textit{term} = \textit{term} \mid \perp$$
$$\textit{literal} ::= \textit{atom} \mid \neg \textit{atom}$$
$$\begin{aligned} \varphi ::= & \textit{literal} \mid \neg \varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \\ & \mid \varphi \rightarrow \varphi \mid \varphi \leftrightarrow \varphi \\ & \mid \forall x \varphi \mid \exists x \varphi \end{aligned}$$

The key point of translation:  $P \rightarrow Q \equiv \neg P \vee Q$

Vampire is based on **refutation proof**.

(It is **proof by contradiction**.)

# Horn Clause and Notation

A **first-order clause** is a  $\forall$ -quantified disjunction of literals.

► **Horn clause:** 0 or 1 positive “head” literals

$$\text{► } \forall x_1 \dots x_k. L_1 \vee L_2 \vee \dots$$

► **definite clause:** exactly 1 positive literal

$$\text{► } \forall x_1 \dots x_k. \neg A_1 \vee \neg A_2 \vee \dots \vee B$$

$$\text{► } \forall x_1 \dots x_k. A_1 \wedge A_2 \wedge \dots \rightarrow B$$

► **goal clause:** exactly 0 positive literals

$$\text{► } \forall x_1 \dots x_k. \neg A_1 \vee \neg A_2 \vee \dots \vee \neg A_n$$

$$\text{► } \forall x_1 \dots x_k. A_1 \wedge A_2 \wedge \dots \wedge A_n \rightarrow \perp$$

each  $L_i$  are Literal and each  $A_i$  and  $B$  are Atom.  
equations  $s = t$  are treated as implicitly symmetric.

# Vampire Core Calculus

---

# Vampire Core Calculus

$$\frac{A \vee C \quad \neg A' \vee D}{\sigma(C \vee D)} \text{ RESOLUTION}$$

$$\frac{A \vee A' \vee C}{\sigma(A \vee C)} \text{ FACTORING}$$

$$\frac{l = r \vee C \quad A[l'] \vee D}{\sigma(A[r] \vee C \vee D)} \text{ SUPERPOSITION} \quad \frac{s \neq t \vee C}{\sigma(C)} \text{ EQ. RESOLUTION}$$

$\sigma : \text{Var} \rightarrow \text{Term}$

$\text{mgu}(A, A')$  in Resolution and Factoring.

$\text{mgu}(l, l')$  in Superposition.

$\text{mgu}(s, t)$  in Equality Resolution.

$A, A[l']$  are atoms.

$C, D$  are clauses.

$l, l', r, s, t$  are terms.

# Example 1 (Vector Spaces)

```
1.  $\forall x. ze + x = x$  (neut1)
2.  $\forall x. (-x) + x = ze$  (neg1)
3.  $\forall x y z. (x + y) + z = x + (y + z)$  (assoc)
4.  $v = u + v$  (e)
5.  $ze \neq u$  (negated goal)
6.  $\forall x y. (-x) + (x + y) = ze + y$  (superposition 3, 2)
7.  $\forall x. v + x = u + (v + x)$  (superposition 3, 4)
8.  $\forall x y. (-x) + (x + y) = y$  (superposition 6, 1)
9.  $\forall x. (-(-x)) + ze = x$  (superposition 8, 2)
10.  $\forall x y. y + x = (-(-y)) + x$  (superposition 8, 8)
11.  $\forall x. x + ze = x$  (superposition 9, 10)
12.  $\forall x. ze = x + (-x)$  (superposition 2, 10)
13.  $ze = u + ze$  (superposition 7, 12)
14.  $ze = u$  (superposition 13, 11)
15. false (resolution 14, 5)

-- Vampire Original Output
1. 'vec._+_('vec.ze',X0) = X0
2. 'vec._+_('vec.__'(X0),X0) = 'vec.ze'
3. 'vec._+_('vec._+'(X0,X1),X2) = 'vec._+'(X0,'vec._+'(X1,X2))
4. v = 'vec._+'(u,v)
5. 'vec.ze' != u
6. 'vec._+_('vec.__'(X0),'vec._+'(X0,X1)) = 'vec._+'('vec.ze',X1)
7. 'vec._+'(v,X0) = 'vec._+'(u,'vec._+'(v,X0))
8. 'vec._+'('vec.__'(X0),'vec._+'(X0,X1)) = X1
9. 'vec._+'('vec._+'('vec.__'(X0)), 'vec.ze') = X0
10. 'vec._+'(X1,X0) = 'vec._+'('vec._+'('vec.__'(X1)),X0)
11. 'vec._+'(X0,'vec.ze') = X0
12. 'vec.ze' = 'vec._+'(X0,'vec.__'(X0))
13. 'vec.ze' = 'vec._+'(u,'vec.ze')
14. 'vec.ze' = u
15. $false
```

# Example 1 (Vector Spaces)

$$2 \quad \forall x. (-x) + x = ze \quad (l = r)$$

$$3 \quad \forall x y z. (x + y) + z = x + (y + z) \quad (A[l'])$$

**Sup. 3, 2**

$$\frac{l = r \quad A[l']}{\sigma(A[r])} \text{ SUP.}$$

$$\sigma = \{x \mapsto -x, y \mapsto x\}$$

$$\frac{(-x) + x = ze \quad ((-x) + x) + z = (-x) + (x + z)}{ze + z = (-x) + (x + z)} \text{ SUP.}$$

# Vampire to Agda

---

# A classical proof to intuitionistic proof

We restrict Vampire to Horn Clauses and show  $\Gamma \vdash G$ .

Assuming that this is provable, and Vampire obtains a classical proof by refutation showing  $\Gamma, \neg G \vdash \perp$

- ▶  $\Gamma, \neg G \vdash \perp$
- ▶  $\Gamma, G \rightarrow \perp \vdash \perp$
- ▶  $\Gamma, G \rightarrow G \vdash G$  (replacing occurrences of  $\perp$  with  $G$ )

$\Gamma$  is “a set of definite clauses”,  $G$  is “ground goal atom”.

# Lemmas

- ▶ Lemma 1: For every VCC rule, if its premises are Horn, then its conclusion is also Horn.  
(Translate Horn Clause to Horn Implication.)
- ▶ Lemma 2: Horn Vampire Core Calculus (HVCC) is intuitionistically valid.
- ▶ Lemma 3: Let  $\Gamma$  be definite clauses and  $G$  **be a ground atom**. If  $\Gamma, G \rightarrow \perp \vdash \perp$  is derivable in HVCC, then  $\Gamma, G \rightarrow G \vdash G$  is also derivable in HVCC.

Vampire Core Calculus = VCC, Horn VCC = HVCC

# Proof of Lemma 1

**VCC**

$$\frac{A \vee C \quad \neg A' \vee D}{\sigma(C \vee D)} \text{ RES.}$$

$$\frac{A \vee A' \vee C}{\sigma(A \vee C)} \text{ FAC.}$$

$$\frac{s \neq t \vee C}{\sigma(C)} \text{ EQ.RES.}$$

$$\frac{l = r \vee C \quad A[l'] \vee D}{\sigma(A[r] \vee C \vee D)} \text{ SUP.}$$

$$\frac{\neg P \vee A \quad \neg A' \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg Q \vee \boxed{B})} \text{ RES.}$$

$$\frac{\neg A \vee \neg A' \vee \neg P \vee \boxed{B}}{\sigma(\neg A \vee \neg P \vee \boxed{B})} \text{ FAC.}$$

$$\frac{s \neq t \vee \neg P \vee \boxed{A}}{\sigma(\neg P \vee \boxed{A})} \text{ EQ.RES.}$$

$$\frac{\neg P \vee l = r \quad \neg A[l'] \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg A[r] \vee \neg Q \vee \boxed{B})} \text{ SUP.L}$$

$$\frac{\neg P \vee l = r \quad \neg Q \vee \boxed{A[l']}}{\sigma(\neg P \vee \neg Q \vee \boxed{A[r]})} \text{ SUP.R}$$

$P, Q$  = premises;  $A, B$  = atoms;  $C, D$  = clauses;

$\text{mgu}(A, A'), \text{mgu}(l, l'), \text{mgu}(s, t)$

$\boxed{\phantom{x}}$  = positive atom.

$\neg P = \neg(A_1 \wedge A_2 \wedge \dots), C = \neg A_1 \vee \neg A_2 \vee \dots$  (Horn and De Morgan)

# Horn Vampire Core Calculus (HVCC)

## HVCC

$$\frac{\neg P \vee A \quad \neg A' \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg Q \vee \boxed{B})} \text{RES.}$$

$$\frac{\neg A \vee \neg A' \vee \neg P \vee \boxed{B}}{\sigma(\neg A \vee \neg P \vee \boxed{B})} \text{FAC.}$$

$$\frac{s \neq t \vee \neg P \vee \boxed{A}}{\sigma(\neg P \vee \boxed{A})} \text{EQ.RES.}$$

$$\frac{\neg P \vee l = r \quad \neg A[l'] \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg A[r] \vee \neg Q \vee \boxed{B})} \text{SUP.L}$$

$$\frac{\neg P \vee l = r \quad \neg Q \vee \boxed{A[l']}}{\sigma(\neg P \vee \neg Q \vee \boxed{A[r]})} \text{SUP.R}$$

$$\frac{P \rightarrow A \quad A' \wedge Q \rightarrow \boxed{B}}{\sigma(P \wedge Q \rightarrow \boxed{B})} \text{RES.}$$

$$\frac{A \wedge A' \wedge P \rightarrow \boxed{B}}{\sigma(A \wedge P \rightarrow \boxed{B})} \text{FAC.}$$

$$\frac{s = t \wedge P \rightarrow \boxed{A}}{\sigma(P \rightarrow \boxed{A})} \text{EQ.RES.}$$

$$\frac{P \rightarrow l = r \quad A[l'] \wedge Q \rightarrow \boxed{B}}{\sigma(P \wedge A[r] \wedge Q \rightarrow \boxed{B})} \text{SUP.L}$$

$$\frac{P \rightarrow l = r \quad Q \rightarrow \boxed{A[l']}}{\sigma(P \wedge Q \rightarrow \boxed{A[r]})} \text{SUP.R}$$

$P, Q$  = premises;  $A, B$  = atoms;  $\text{mgu}(A, A')$ ,  $\text{mgu}(l, l')$ ,  $\text{mgu}(s, t)$

definite clause:  $\neg A_1 \vee \neg A_2 \vee \dots \vee B \implies A_1 \wedge A_2 \wedge \dots \rightarrow B$

goal clause:  $\neg A_1 \vee \neg A_2 \vee \dots \vee \neg A_n \implies A_1 \wedge A_2 \wedge \dots \wedge A_n \rightarrow \perp$

# Proof of Lemma 2

## HVCC

$$\frac{P \rightarrow A \quad A' \wedge Q \rightarrow \boxed{B}}{\sigma(P \wedge Q \rightarrow \boxed{B})} \text{RES.}$$

$$\frac{A \wedge A' \wedge P \rightarrow \boxed{B}}{\sigma(A \wedge P \rightarrow \boxed{B})} \text{FAC.}$$

$$\frac{s = t \wedge P \rightarrow \boxed{A}}{\sigma(P \rightarrow \boxed{A})} \text{EQ.RES.}$$

$$\frac{P \rightarrow l = r \quad A[l'] \wedge Q \rightarrow \boxed{B}}{\sigma(P \wedge A[r] \wedge Q \rightarrow \boxed{B})} \text{SUP.L}$$

$$\frac{P \rightarrow l = r \quad Q \rightarrow \boxed{A[l']}}{\sigma(P \wedge Q \rightarrow \boxed{A[r]})} \text{SUP.R}$$

## Agda

$$\frac{t_1 : P \rightarrow A \quad t_2 : A \wedge Q \rightarrow B}{(\lambda(p, q). t_2 (t_1 p, q)) : P \wedge Q \rightarrow B} \text{RES.}$$

$$\frac{t : A \wedge A \wedge P \rightarrow B}{(\lambda(a, p). t (a, a, p)) : A \wedge P \rightarrow B} \text{FAC.}$$

$$\frac{t : s = s \wedge P \rightarrow A}{(\lambda p. t \text{ refl } p) : P \rightarrow A} \text{EQ.RES.}$$

$$\frac{t_1 : P \rightarrow l = r \quad t_2 : A[l] \wedge Q \rightarrow B}{(\lambda(p, a, q). t_2 ((\text{subst } (\text{sym } (t_1 p)) a), q)) : P \wedge A[r] \wedge Q \rightarrow B} \text{SUP.L}$$

$$\frac{t_1 : P \rightarrow l = r \quad t_2 : Q \rightarrow A[l]}{(\lambda(p, q). \text{subst } (t_1 p) (t_2 q)) : P \wedge Q \rightarrow A[r]} \text{SUP.R}$$

$\text{refl} : l = l$ ,  $\text{sym} : l = r \rightarrow r = l$ ,  $\text{subst} : l = r \rightarrow A[l] \rightarrow A[r]$ .

# An example of applying Lemma 3

$$t_1 : \forall x. f(x) = x, \quad t_2 : a = f(c), \quad t_3 : b = c, \quad t_4 : a = b \rightarrow \boxed{\perp} \vdash \boxed{\perp}$$

$$\frac{\frac{\frac{b = c}{f(c) = c} \quad \frac{\frac{a = f(c) \quad a = b \rightarrow \boxed{\perp}}{f(c) = b \rightarrow \boxed{\perp}} \text{ SUP.L}}{c = b \rightarrow \boxed{\perp}} \text{ SUP.L}}{c = c \rightarrow \boxed{\perp}} \text{ SUP.L}}{\boxed{\perp}} \text{ EQ.RES.}$$

replace each  $\boxed{\perp}$  by  $a = b$

# Thesis Proposal

---

# Thesis Proposal

Formalize and verify the translation from Vampire to Agda.

- ▶ Formalize HVCC by deep embedding.
- ▶ Prove Lemmas 2, 3.

# References

- ▶ Artjoms Šinkarovs and Michael Rawson. “When Agda met Vampire.”
- ▶ Laura Kovács and Andrei Voronkov. “Vampire Guide Lectures.”
- ▶ Giles Reger. “The Vampire Journey. (Youtube)”
- ▶ Alan J.A. Robinson, Andrei Voronkov. “Paramodulation-Based Theorem Proving”